

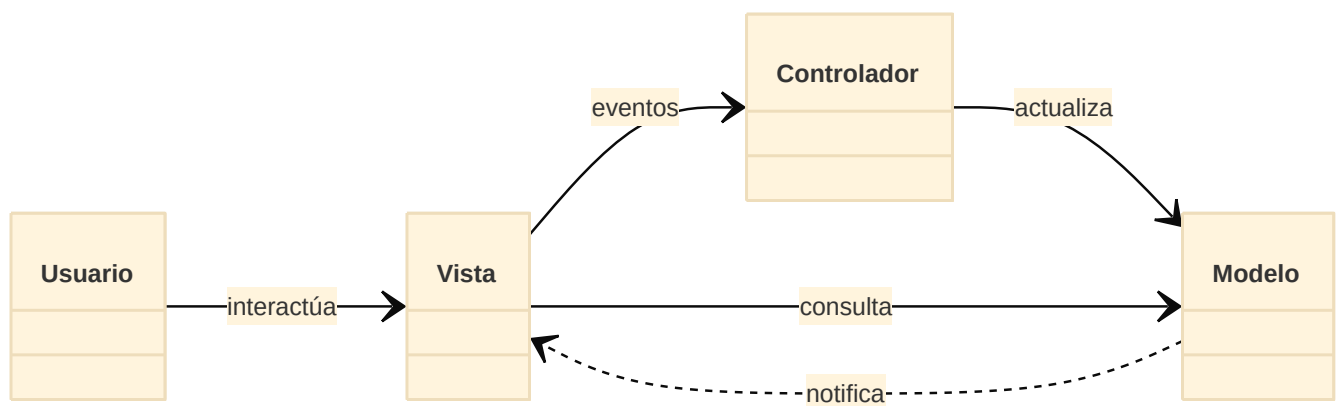
Patrones

Patrón MVC

El patrón model-view-controller es un patrón de diseño software por capas. Se divide en:

- **Modelo:** representa la capa de datos y la lógica de negocio de la aplicación. Contiene la representación interna de los datos y las operaciones que se pueden realizar sobre ellos. Notifica a las vistas sobre cambios en los datos, permitiendo que la interfaz de usuario se actualice en consecuencia.
- **Vista:** es responsable de presentar la interfaz de usuario al usuario final. Muestra los datos provenientes del modelo y se actualiza automáticamente cuando los datos cambian. Generalmente, la vista en MVC puede contener cierta cantidad de lógica de presentación, aunque se espera que esta lógica sea mínima.
- **Controlador:** maneja las interacciones del usuario y actúa como intermediario entre la vista y el modelo. Contiene lógica de presentación y lógica de negocio relacionada con las interacciones del usuario. Actualiza el Modelo en respuesta a las acciones del usuario y actualiza la Vista en consecuencia.

UML de MVC



Comunicación

Sigue un flujo unidireccional. El controlador maneja las acciones del usuario y actualiza el modelo y la vista según sea necesario. La vista observa al modelo y se actualiza automáticamente cuando los datos cambian.

Ventajas

- Facilita la reutilización del código al separar la lógica de presentación y la lógica de negocio.
- Mejora la modularidad y el mantenimiento del código al reducir el acoplamiento entre los componentes.

Inconvenientes

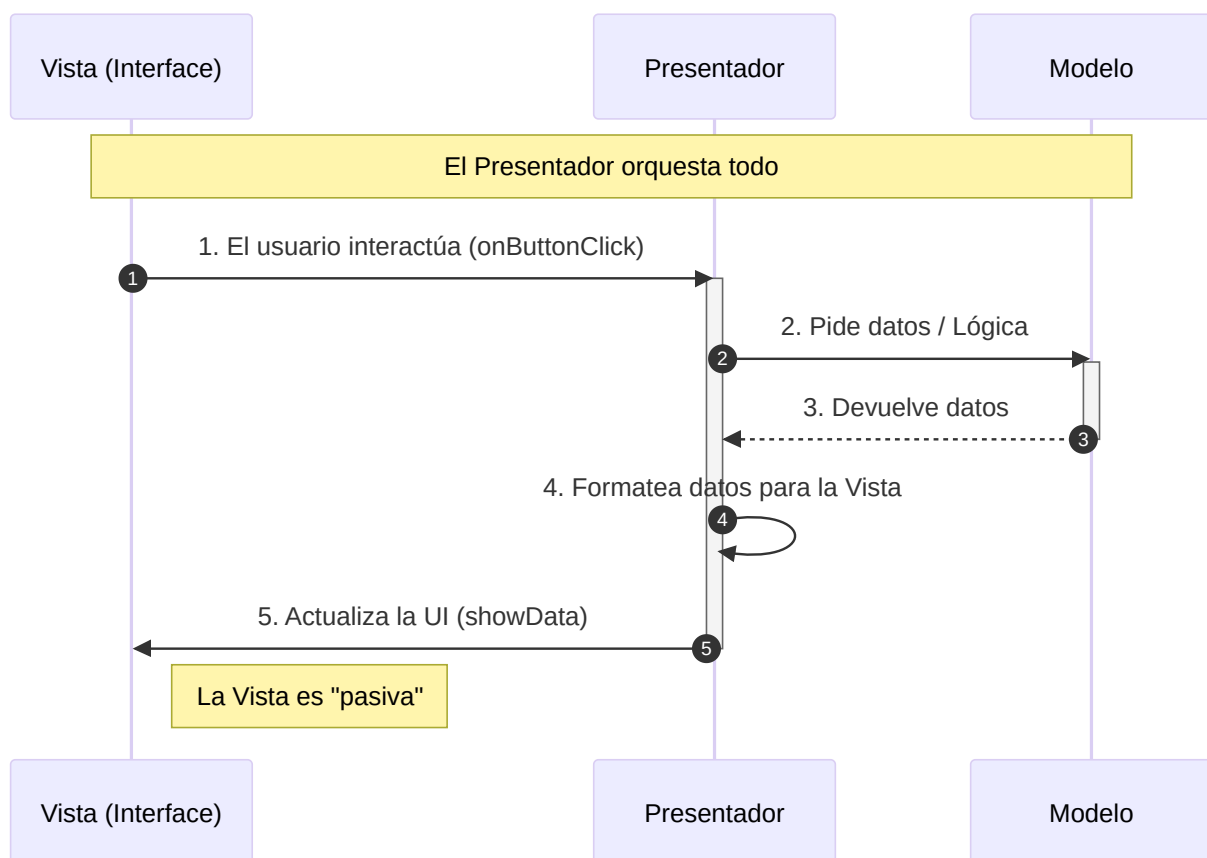
- La separación de responsabilidades puede ser difusa, especialmente en el controlador, que puede volverse complejo.
- La comunicación unidireccional puede complicar la comprensión del flujo de datos en comparación con patrones más modernos como MVP.

Patrón MVP (Model-View-Presenter)

Es una evolución del patrón MVC, diseñado para mejorar la separación de preocupaciones y facilitar las pruebas unitarias en el desarrollo de software. Es también un patrón de diseño por capas, model-view-presenter. Se divide en:

- **Modelo:** representa la capa de datos y la lógica de negocio de la aplicación. Contiene la representación interna de los datos y las operaciones que se pueden realizar sobre ellos. La independencia de la interfaz de usuario permite que el modelo pueda ser probado de manera aislada sin depender de la lógica de presentación.
- **Vista:** es responsable de presentar la interfaz de usuario al usuario final. Difiere del MVC en que es pasiva y carece de lógica de presentación significativa. Al no contener lógica de control es más fácil de probar, ya que su interacción con el presentador es principalmente a través de interfaces.
- **Presentador:** actúa como un intermediario entre la vista y el modelo. Contiene la lógica de presentación y se encarga de manejar las interacciones del usuario. La comunicación entre la vista y el presentador es bidireccional, permitiendo que el presentador actualice la vista y maneje las acciones del usuario. Al tener la lógica de presentación separada, facilita las pruebas unitarias y mejora la modularidad del código.

UML de MVP



Comunicación

Está diseñada para minimizar el acoplamiento. El presentador, al manejar las interacciones del usuario, actualiza tanto el modelo como la vista según sea necesario. Este enfoque de comunicación facilita la mantenibilidad y flexibilidad.

Ventajas

- Separación clara de responsabilidades entre el Modelo, la Vista y el Presentador.
- Facilita las pruebas unitarias al tener la lógica de presentación independiente.
- Mejora la modularidad y el mantenimiento del código al reducir el acoplamiento entre los componentes.

Inconvenientes

- Puede generar una mayor cantidad de código debido a la estricta separación de responsabilidades.
- La comunicación bidireccional entre la vista y el presentador puede resultar compleja en algunos casos.

Patrón de Diseño BLoC (Business Logic Component)

Definición y Propósito

El patrón **BLoC** es un sistema de gestión de estado basado en la **programación reactiva**. Fue creado por Google para Flutter, aunque su concepto es aplicable a otras tecnologías.

- **Objetivo:** Separar la lógica de negocio (cómo funcionan los datos) de la interfaz de usuario (cómo se ven los datos).
- **Principio:** Sigue el principio de **Responsabilidad Única**. La UI no debe contener lógica de cálculo ni peticiones a bases de datos.

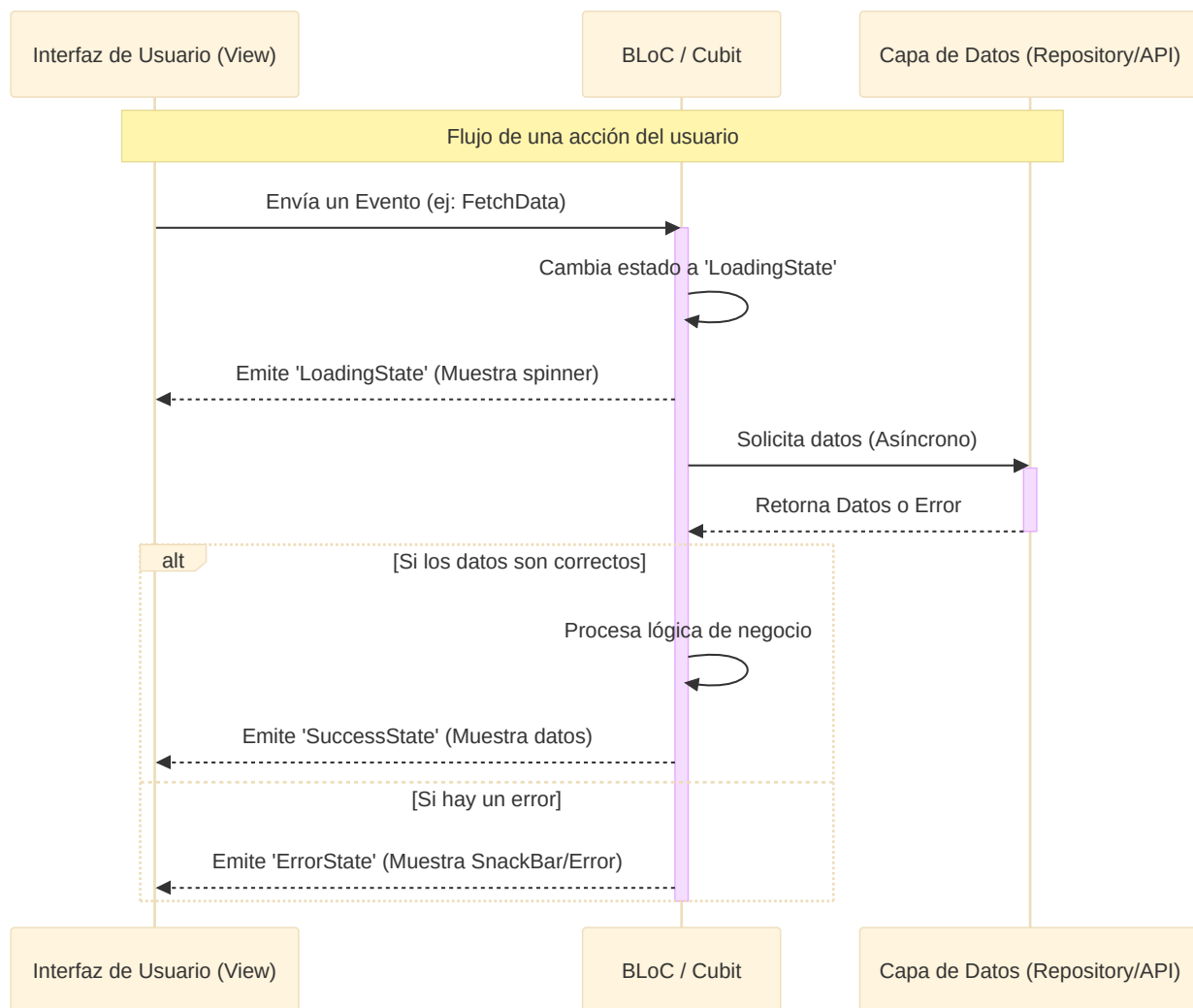
Arquitectura y Flujo de Datos

BLoC funciona como un intermediario que utiliza **Streams** (flujos de datos). El flujo es estrictamente unidireccional.

Los 3 pilares del flujo:

1. **Events (Eventos):** Son las entradas (**Inputs**). Representan cualquier acción que el usuario realice (ej: `ClickBotonLogin`).
2. **BLoC (Cerebro):** Recibe el evento, procesa la información (puede llamar a un Repositorio o API) y decide qué estado emitir.
3. **States (Estados):** Son las salidas (**Outputs**). Representan una "foto" de la aplicación en un momento dado (ej: `Cargando`, `Error`, `Autenticado`).

UML de BLoC



Ventajas para el Desarrollador

- **Mantenibilidad:** Es fácil localizar errores porque la lógica está aislada.
- **Testabilidad:** Puedes probar la lógica del BLoC sin necesidad de ejecutar un emulador o probar la interfaz.
- **Escalabilidad:** Ideal para equipos grandes donde unos trabajan en el diseño y otros en los datos.

Ejemplo de Flujo (Examen Práctico)

Si te piden describir el flujo de un **Contador**:

1. **Evento:** `IncrementarCounter`.
2. **BLoC:** Recibe el evento, toma el valor actual (ej: 5), le suma 1.
3. **Estado:** Emite un nuevo estado `CounterValue(6)`.
4. **UI:** El `BlocBuilder` recibe el 6 y actualiza el texto en pantalla.

Widgets Clave (Librería `flutter_bloc`)

En el examen de interfaces, es vital saber cuándo usar cada widget:

Widget	Función Principal
BlocProvider	Injecta el BLoC en el árbol de widgets para que sus hijos puedan acceder a él.

Widget	Función Principal
BlocBuilder	Escucha los cambios de estado y reconstruye la interfaz (devuelve un widget).
BlocListener	Escucha cambios de estado pero no dibuja nada . Se usa para navegar, mostrar alertas (SnackBar) o diálogos.
BlocConsumer	Una mezcla de Builder y Listener. Útil cuando necesitas redibujar la pantalla y mostrar un mensaje al mismo tiempo.

Patrón Model View Viewmodel

El patrón Model-View-ViewModel (MVVM) es un patrón de arquitectura de software diseñado para facilitar la separación de la interfaz de usuario (la parte visual) de la lógica de negocio y el comportamiento de la aplicación.

Estructura de MVVM

El patrón se divide en tres componentes principales que interactúan entre sí para mantener el código organizado y testeable:

Model: Representa la capa de datos y la lógica de negocio. Es el encargado de gestionar el acceso a datos (bases de datos, APIs) y las reglas de negocio. No sabe nada de la interfaz de usuario ni del ViewModel. Solo emite datos.

View: Es la estructura, el diseño y la apariencia de lo que el usuario ve en pantalla. Define la disposición de los elementos visuales. Recibe las interacciones del usuario (clics, gestos). Se "conecta" al ViewModel mediante el Data Binding. Lo ideal es que la vista no tenga lógica (cero "code-behind").

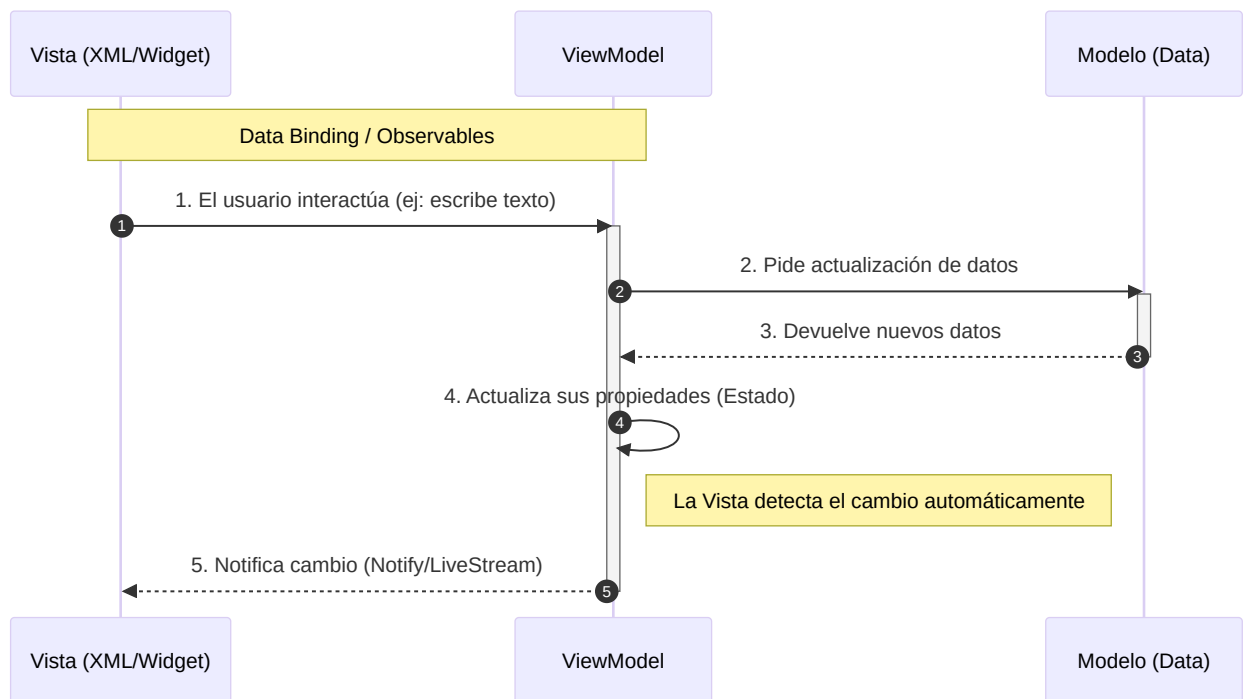
ViewModel: Actúa como un mediador entre el Modelo y la Vista. Expone los datos del Modelo de una manera que la Vista pueda consumir fácilmente. Transforma los datos crudos del modelo en información visual. Envía comandos a la Vista y reacciona a los cambios en el Modelo. No conoce los elementos específicos de la UI (no sabe qué es un botón o un label), solo maneja estados.

El Flujo de Comunicación

El éxito de MVVM reside en cómo se comunican estas partes:

1. **Data Binding:** La Vista está "atada" a las propiedades del ViewModel. Si el ViewModel cambia un valor, la Vista se actualiza automáticamente.
2. **Commands (Comandos):** Cuando el usuario interactúa con la Vista, esta invoca un comando en el ViewModel para ejecutar una acción.
3. **Notifications:** El ViewModel suele implementar interfaces (como `INotifyPropertyChanged` en .NET) para avisar a la Vista que algo ha cambiado.

Diagrama UML de MVVM



Ventajas de usar MVVM

- **Testeabilidad:** Puedes probar la lógica del ViewModel sin necesidad de abrir una interfaz gráfica.
- **Mantenibilidad:** Separar la UI de la lógica permite que los diseñadores trabajen en la Vista mientras los desarrolladores trabajan en el ViewModel.
- **Reutilización:** Un mismo ViewModel podría ser utilizado por diferentes Vistas (por ejemplo, una versión móvil y una de escritorio).

Patrón Provider

Definición y Propósito

El patrón **Provider** es un patrón de diseño utilizado principalmente para la **gestión de estados** y la **inyección de dependencias**. Su objetivo principal es permitir que los datos (el "estado") estén disponibles para múltiples componentes en un árbol jerárquico sin necesidad de pasarlos manualmente a través de constructores o propiedades (técnica conocida como *Prop Drilling*).

2. Componentes Clave

Para que el patrón funcione, se basa en tres roles fundamentales:

Componente	Función
ChangeNotifier (El Modelo)	Es la clase que contiene los datos y la lógica. Notifica a los demás cuando algo cambia.
ChangeNotifierProvider (El Almacén)	Es el widget que "envuelve" a una parte de la aplicación para proveer el modelo a sus hijos.
Consumer / Provider.of (El Cliente)	Es el mecanismo que usan los widgets hijos para "escuchar" los cambios y redibujarse.

Funcionamiento del Flujo de Datos

El flujo sigue una estructura unidireccional para mantener la predictibilidad:

1. **Estado centralizado:** El dato vive en un solo lugar (el Provider).
2. **Notificación:** Cuando el dato cambia, se dispara un evento (normalmente el método `notifyListeners()`).
3. **Suscripción:** Los widgets que están "escuchando" ese Provider específico se reconstruyen automáticamente con el nuevo valor.

Ventajas y Desventajas

Ventajas

- **Desacoplamiento:** La lógica de negocio se separa de la interfaz de usuario.
- **Eficiencia:** Solo se repintan los widgets que consumen el dato, no toda la pantalla.
- **Código limpio:** Evita el *Prop Drilling* (pasar datos por niveles innecesarios).

Desventajas

- **Curva de aprendizaje:** Puede ser confuso al principio entender cuándo usar `watch` , `read` o `select` .
- **Sobrecarga en apps simples:** Para aplicaciones muy pequeñas, puede añadir complejidad innecesaria.

Ejemplo de Implementación (Sintaxis Flutter/Dart)

Paso A: Crear el Modelo (State)

```
class Contador extends ChangeNotifier {  
  int _valor = 0;  
  int get valor => _valor;  
  
  void incrementar() {  
    _valor++;  
    notifyListeners(); // Esto avisa a la UI que debe actualizarse  
  }  
}
```

Paso B: Proveer el Modelo

Se coloca en la raíz o sobre el árbol que necesite los datos.

```
ChangeNotifierProvider(  
  create: (context) => Contador(),  
  child: MyApp(),  
)
```

Paso C: Consumir los datos

```
// En el widget hijo  
final contador = Provider.of<Contador>(context);
```

```
return Text('Valor actual: ${contador.valor}');
```

Diferencia entre Watch, Read y Select (Tips de examen)

- `context.watch<T>()`: El widget se queda "escuchando". Si el dato cambia, el widget se reconstruye. Se usa en el método `build`.
- `context.read<T>()`: Accede al dato pero no escucha cambios. Ideal para funciones (ej. el `onPressed` de un botón).
- `context.select<T, R>()`: Permite escuchar solo una parte específica del modelo para evitar reconstrucciones innecesarias.

